

L'épreuve comporte 37 questions réparties en cinq parties après les préliminaires. On pourra réutiliser tout résultat d'une question antérieure, même non démontré. Le but est d'étudier les relations entre automates reconnaissant un même langage grâce à la notion de morphisme d'automates.

Préliminaires

Concernant la programmation

Les fonctions seront écrites en OCaml. On pourra utiliser les fonctions définies aux questions précédentes et définir des fonctions auxiliaires. Il n'est pas nécessaire de justifier le code, sauf demande explicite, ni de vérifier dans le code les hypothèses imposées aux paramètres. Un même identifiant en italique et à chasse fixe désigne la même entité, respectivement du point de vue mathématique et informatique.

Définition mathématique d'un automate

Dans tout le sujet, un automate est un automate fini déterministe complet sur $\{a, b\}$, c'est-à-dire un quadruplet $\mathcal{A} = \langle Q, i, \delta, F \rangle$ avec $i \in Q$, $\delta : Q \times \{a, b\} \rightarrow Q$ et $F \subseteq Q$. On note ε le mot vide et $\delta^* : Q \times \{a, b\}^* \rightarrow Q$ l'extension définie par

$$\delta^*(q, \varepsilon) = q, \quad \delta^*(q, \sigma w) = \delta^*(\delta(q, \sigma), w).$$

L'automate est représenté par un graphe orienté $G = (S, A)$ avec $S = Q$: un arc $p \rightarrow q$ porte la lettre σ si $\delta(p, \sigma) = q$. Une flèche entrante indique l'état initial ; un double cercle indique un état final.

Représentation en OCaml

Les états sont numérotés de 0 à $|Q| - 1$, l'état initial porte toujours le numéro 0. Un automate est un triplet (n, delta, f) :

- `n` : `int` est le nombre d'états ;
- `delta` : `(int * int) array` contient les couples $(\delta(q, a), \delta(q, b))$;
- `f` : `bool array` indique les états finaux.

Les deux tableaux sont de longueur n .

```
type automate = int * (int * int) array * bool array
type morphisme = int array
```

Ainsi, `let (n, delta, f) = aut in ...` extrait les composantes, `let (succ_a, succ_b) = delta.(q) in ...` les successeurs de q , et `if f.(q) then ...` teste si q est final. On rappelle que `List.length` donne la longueur d'une liste, `Array.make n x` crée un tableau initialisé à x , `Array.copy` copie un tableau et `Array.length` donne sa longueur. `Array.make_matrix n m x` crée une matrice $n \times m$ dont les lignes sont indépendantes. On utilisera aussi `List.iter f l`, qui applique f à chaque élément de l , dans l'ordre.

I – Premiers exemples

1. Donner, sans preuve, une description courte en langue française du langage reconnu par $\mathcal{A}_1$ (figure 1).
2. Donner, sans preuve, une description courte en langue française du langage reconnu par $\mathcal{A}_2$ (figure 2).
3. Donner, sans preuve, une expression régulière du langage reconnu par $\mathcal{A}_1$.
4. Donner, sans preuve, une expression régulière du langage reconnu par $\mathcal{A}_2$.
5. Écrire en OCaml, sans justification, une instance du type `automate` correspondant à $\mathcal{A}_2$.

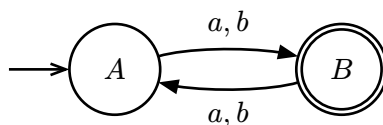


Figure 1 — Automate $\mathcal{A}_1$

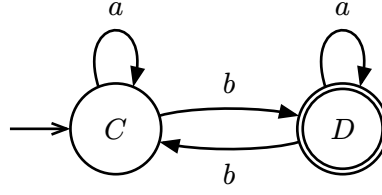


Figure 2 — Automate $\mathcal{A}_2$

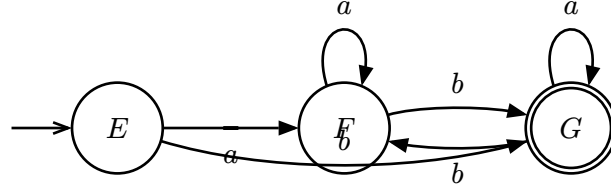


Figure 3 — Automate $\mathcal{A}_3$

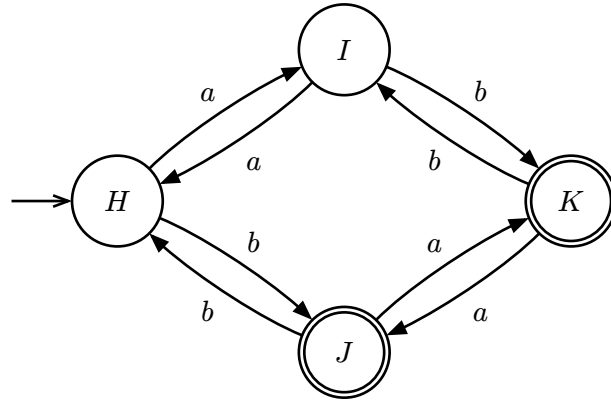


Figure 4 — Automate $\mathcal{A}_4$

II – États accessibles d'un automate

6. Écrire une fonction `numero : int -> int list -> int array` qui, à partir d'un entier n et d'une liste A d'entiers entre 0 et $n - 1$, renvoie un tableau T de taille n : $T[i] = -1$ si i est absent de A , et $T[i]$ est l'indice de l'une de ses occurrences sinon. Par exemple, `numero 5 [3;2;0]` peut renvoyer `[2;-1;1;0;-1]`.

Un état q de $\mathcal{A} = \langle Q_{\mathcal{A}}, i_{\mathcal{A}}, \delta_{\mathcal{A}}, F_{\mathcal{A}} \rangle$ est accessible s'il existe $w \in \{a, b\}^*$ tel que $\delta_{\mathcal{A}}^*(i_{\mathcal{A}}, w) = q$. L'état initial est donc toujours accessible. Si Q' est l'ensemble des états accessibles, la partie accessible est $\mathcal{A}' = \langle Q', i_{\mathcal{A}}, \delta', F_{\mathcal{A}} \cap Q' \rangle$, où δ' est la restriction de $\delta_{\mathcal{A}}$ à $Q' \times \{a, b\}$. Un automate est accessible si tous ses états le sont.

7. Écrire `etats_accessibles : automate -> int list` renvoyant les états accessibles dans l'ordre de leur première rencontre lors d'un parcours en profondeur depuis l'état initial, sans doublons. Donner sa complexité.
8. Écrire `partie_accessible : automate -> automate` construisant la partie accessible de l'automate donné. On pourra réemployer les questions 6 et 7.

III – Morphismes d'automates

Soient $\mathcal{A} = \langle Q_{\mathcal{A}}, i_{\mathcal{A}}, \delta_{\mathcal{A}}, F_{\mathcal{A}} \rangle$ et $\mathcal{B} = \langle Q_{\mathcal{B}}, i_{\mathcal{B}}, \delta_{\mathcal{B}}, F_{\mathcal{B}} \rangle$ deux automates. Un morphisme $\varphi : \mathcal{A} \rightarrow \mathcal{B}$ est une application $\varphi : Q_{\mathcal{A}} \rightarrow Q_{\mathcal{B}}$ satisfaisant :

$$\varphi \text{ est surjective,} \quad (1)$$

$$\varphi(i_{\mathcal{A}}) = i_{\mathcal{B}}, \quad (2)$$

$$\forall q \in Q_{\mathcal{A}}, \forall \sigma \in \{a, b\}, \quad \varphi(\delta_{\mathcal{A}}(q, \sigma)) = \delta_{\mathcal{B}}(\varphi(q), \sigma), \quad (3)$$

$$\forall q \in Q_{\mathcal{A}}, \quad q \in F_{\mathcal{A}} \Leftrightarrow \varphi(q) \in F_{\mathcal{B}}. \quad (4)$$

En OCaml, φ est représenté par le tableau $[\varphi(q)]_{q \in Q_{\mathcal{A}}}$, de type `morphisme`, de longueur $|Q_{\mathcal{A}}|$ et à valeurs entre 0 et $|Q_{\mathcal{B}}| - 1$.

III.A – Exemples de morphismes d'automates

9. Recopier et compléter, sans justification, le tableau par des états de $\mathcal{A}_2$ de sorte qu'il représente un morphisme $\varphi : \mathcal{A}_3 \rightarrow \mathcal{A}_2$.

q	$\varphi(q)$
E	
F	
G	

10. Donner, sans justification, un morphisme de $\mathcal{A}_4$ vers $\mathcal{A}_2$.
 11. Montrer qu'il n'existe pas de morphisme de $\mathcal{A}_1$ vers $\mathcal{A}_2$.
 12. Montrer qu'il n'existe pas de morphisme de $\mathcal{A}_5$ (figure 5) vers $\mathcal{A}_2$.

III.B – Propriétés des morphismes d'automates

13. Montrer que deux automates acceptent le même langage dès qu'il existe un morphisme de l'un vers l'autre.
 14. Montrer qu'un morphisme φ entre deux automates de même nombre d'états est bijectif et que φ^{-1} est encore un morphisme. On dit alors que φ est un isomorphisme.
 15. Montrer que la composition de deux morphismes est encore un morphisme.

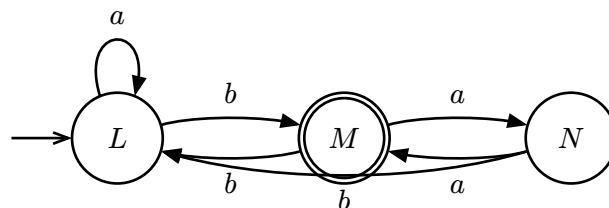


Figure 5 — Automate $\mathcal{A}_5$

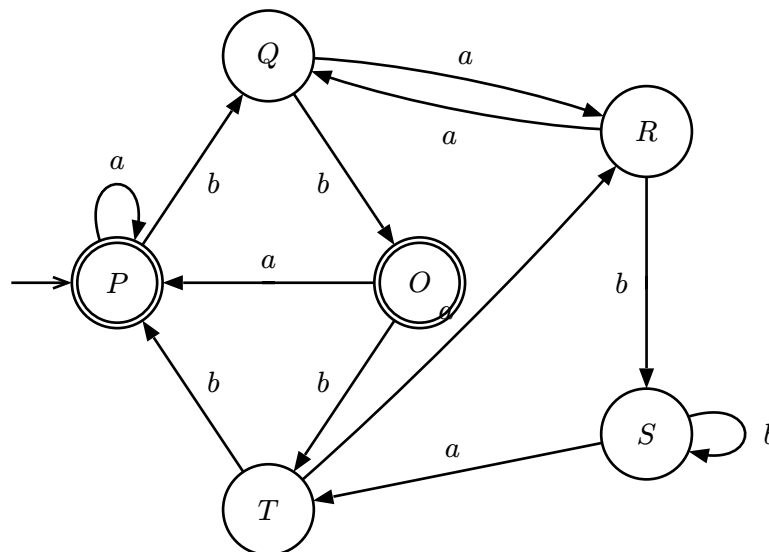


Figure 6 — Automate $\mathcal{A}_6$

III.C – Existence de morphismes entre automates accessibles

16. Montrer que (1) découle de (2), (3) et (4) lorsque les deux automates sont accessibles.

17. Écrire `existe_morphisme : automate -> automate -> bool * morphisme` qui, pour deux automates accessibles, indique s'il existe un morphisme du premier vers le second et en renvoie un lorsqu'il existe. Sinon, le tableau renvoyé est quelconque. On pourra expliquer le principe de l'algorithme avant le code.

IV – Constructions de morphismes d'automates

IV.A – Automate produit

Pour deux automates $\mathcal{A}$ et $\mathcal{A}'$, on définit l'automate produit

$$\mathcal{A} \times \mathcal{A}' = \langle Q_{\mathcal{A}} \times Q_{\mathcal{A}'}, (i_{\mathcal{A}}, i_{\mathcal{A}'}), \delta_{\mathcal{A} \times \mathcal{A}'}, F_{\mathcal{A}} \times F_{\mathcal{A}'} \rangle$$

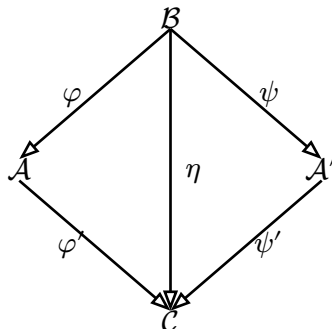
avec, pour chaque couple (q, q') et lettre $\sigma \in \{a, b\}$,

$$\delta_{\mathcal{A} \times \mathcal{A}'}((q, q'), \sigma) = (\delta_{\mathcal{A}}(q, \sigma), \delta_{\mathcal{A}'}(q', \sigma)).$$

18. Dessiner, sans justification, la partie accessible du produit $\mathcal{A}_3 \times \mathcal{A}_4$.
19. Écrire `produit : automate -> automate -> automate` renvoyant le produit des deux automates donnés.
20. Soit (q, q') un état accessible du produit de deux automates acceptant le même langage. Montrer que q est final dans le premier si et seulement si q' est final dans le second.
21. Montrer qu'il existe toujours un morphisme de la partie accessible du produit de deux automates accessibles acceptant le même langage vers chacun de ces automates.

IV.B – Diagramme d'automates

Dans toute cette sous-partie, $\mathcal{A}$, $\mathcal{A}'$ et $\mathcal{B}$ sont accessibles, et $\varphi : \mathcal{B} \rightarrow \mathcal{A}$ et $\psi : \mathcal{B} \rightarrow \mathcal{A}'$ sont des morphismes. On veut construire un automate accessible $\mathcal{C}$ et trois morphismes φ' , ψ' , η :



Pour $(p, q) \in Q_{\mathcal{B}}^2$, on pose $p \equiv q$ s'il existe une suite finie $p = q_0, q_1, \dots, q_k = q$, avec $k \in \mathbb{N}$, telle que

$$\forall 0 \leq j < k, \quad \varphi(q_j) = \varphi(q_{j+1}) \quad \text{ou} \quad \psi(q_j) = \psi(q_{j+1}).$$

22. Montrer que $\equiv$ est une relation d'équivalence sur $Q_{\mathcal{B}}$.
23. Montrer que $p \equiv q$ entraîne $\delta_{\mathcal{B}}(p, \sigma) \equiv \delta_{\mathcal{B}}(q, \sigma)$ pour toute lettre $\sigma \in \{a, b\}$.
24. Montrer que, si $p \equiv q$, alors p est final dans $\mathcal{B}$ si et seulement si q l'est.
- La classe d'un état q est $[q] = \{p \in Q_{\mathcal{B}} \mid q \equiv p\}$. On note ℓ le nombre de classes et $S_0, \dots, S_{\ell-1}$ ces classes, avec $i_{\mathcal{B}} \in S_0$. On définit $\eta : Q_{\mathcal{B}} \rightarrow \{S_0, \dots, S_{\ell-1}\}$ par $\eta(q) = [q]$. En OCaml, la classe S_j est représentée par l'indice j .
25. Construire un automate accessible $\mathcal{C}$ d'ensemble d'états $\{S_0, \dots, S_{\ell-1}\}$ tel que $\eta : \mathcal{B} \rightarrow \mathcal{C}$ soit un morphisme. Justifier.
26. Construire deux morphismes $\varphi' : \mathcal{A} \rightarrow \mathcal{C}$ et $\psi' : \mathcal{A}' \rightarrow \mathcal{C}$, où $\mathcal{C}$ est l'automate de la question 25.
27. Écrire `renomme : int array -> int array` qui renomme un tableau d'entiers positifs ou nuls prenant ℓ valeurs distinctes à l'aide des entiers de 0 à $\ell - 1$. Si le tableau est non vide, le premier élément du résultat doit valoir 0. Par exemple, `renomme [|4;4;5;0;4;5|]` peut renvoyer `[|0;0;1;2;0;1|]`. Préciser la complexité.

28. Écrire `relation : morphisme -> morphisme -> morphisme` qui, à partir de $[\varphi(q)]$ et $[\psi(q)]$, renvoie $[\eta(q)]$, c'est-à-dire un tableau `t` à valeurs entre 0 et $\ell - 1$ tel que $t.(p) = t.(q)$ équivaut à $p \equiv q$, et $t.(0) = 0$.

V – Réduction d'automates

V.A – Existence et unicité

29. Montrer que, si deux automates accessibles $\mathcal{A}$ et $\mathcal{A}'$ acceptent le même langage, on peut construire un automate $\mathcal{C}$ et deux morphismes $\varphi' : \mathcal{A} \rightarrow \mathcal{C}$ et $\psi' : \mathcal{A}' \rightarrow \mathcal{C}$.

30. Déterminer l'automate $\mathcal{C}$ de la question 29 pour $\mathcal{A}_3$ et $\mathcal{A}_4$, et préciser φ' et ψ' .

Soit L un langage régulier et $\mathcal{K}_L$ l'ensemble des automates déterministes complets accessibles qui acceptent L . On note m_L leur plus petit nombre d'états.

31. Montrer que deux automates de $\mathcal{K}_L$ ayant m_L états sont nécessairement isomorphes.

32. Montrer que, pour tout $\mathcal{A} \in \mathcal{K}_L$, il existe un morphisme $\varphi : \mathcal{A} \rightarrow \mathcal{M}_L$, où $\mathcal{M}_L$ est un automate de $\mathcal{K}_L$ à m_L états.

V.B – Construction d'un automate réduit par fusion d'états

Deux états p, q de $\mathcal{A}$ ont été fusionnés dans $\mathcal{A}'$ s'il existe un morphisme $\varphi : \mathcal{A} \rightarrow \mathcal{A}'$ tel que $\varphi(p) = \varphi(q)$ et $|Q_{\mathcal{A}'}| < |Q_{\mathcal{A}}|$.

33. Dessiner un automate $\mathcal{A}_6^{O,P}$ dans lequel les états O et P de $\mathcal{A}_6$ ont été fusionnés. Donner un morphisme $\mathcal{A}_6 \rightarrow \mathcal{A}_6^{O,P}$.

34. Expliquer brièvement pourquoi on ne peut construire $\mathcal{A}_6^{Q,R}$ muni d'un morphisme $\psi : \mathcal{A}_6 \rightarrow \mathcal{A}_6^{Q,R}$ tel que $\psi(Q) = \psi(R)$.

35. Quels états faut-il encore fusionner dans $\mathcal{A}_6^{O,P}$ pour obtenir un automate à trois états $\mathcal{M}_{L_6}$ reconnaissant le même langage que $\mathcal{A}_6$?

36. Soit $\mathcal{A} = \langle Q, i, \delta, F \rangle$ un automate accessible. On appelle $P = (S, A)$ le graphe orienté de sommets $S = Q \times Q$, avec, pour chaque $\sigma \in \{a, b\}$, un arc de (p, q) vers $(\delta(p, \sigma), \delta(q, \sigma))$. Écrire `table_predecesseurs : automate -> bool array array` renvoyant une matrice $n \times n$ dont la case (p, q) vaut `true` si et seulement s'il existe un chemin de (p, q) vers un couple (p_0, q_0) dont exactement un état appartient à F . On essaiera de ne pas dépasser $O(n^2)$.

37. Décrire le principe, le justifier, puis écrire `reduit : automate -> automate` qui renvoie l'automate $\mathcal{M}_L$ associé au langage L de l'automate donné.